

[← Go Back](#)

Library

Recents viewed

ArduinoBLE

[ArduinoMqttClient](#)[WiFiNINA](#)[Arduino SigFox for
MKRFox1200](#)[autowp-mcp2515](#)[Arduino_MKRENV](#)[Arduino_MKRGPS](#)[Arduino_APDS9960](#)[Arduino_MKRIoTCarrier](#)[Home](#) / [Programming](#) / [Library](#) / **ArduinoBLE**

ON THIS PAGE

COMMUNICATION

ArduinoBLE

[ARDUINO](#) [GNU Lesser General Public License v2.1](#) V1.4.1 [Arduino](#) [24/07/2025](#)[Arduino](#) <info@arduino.cc><https://www.arduino.cc/en/Re...> info@arduino.cc

Enables Bluetooth® Low Energy connectivity on the Arduino MKR WiFi 1010, Arduino UNO WiFi Rev2, Arduino Nano 33 IoT, Arduino Nano 33 BLE, Nicla Sense ME and UNO R4 WiFi.

This library supports creating a Bluetooth® Low Energy peripheral & central mode.

GO TO REPOSITORY

Usage/Examples

Compatibility

Releases

This library supports all the Arduino boards that have the hardware enabled for Bluetooth® Low Energy and Bluetooth® 4.0 and above; these include Nano 33 BLE, Arduino NANO 33 IoT, Uno WiFi Rev2, MKR WiFi 1010, Nicla Sense ME.

To use this library

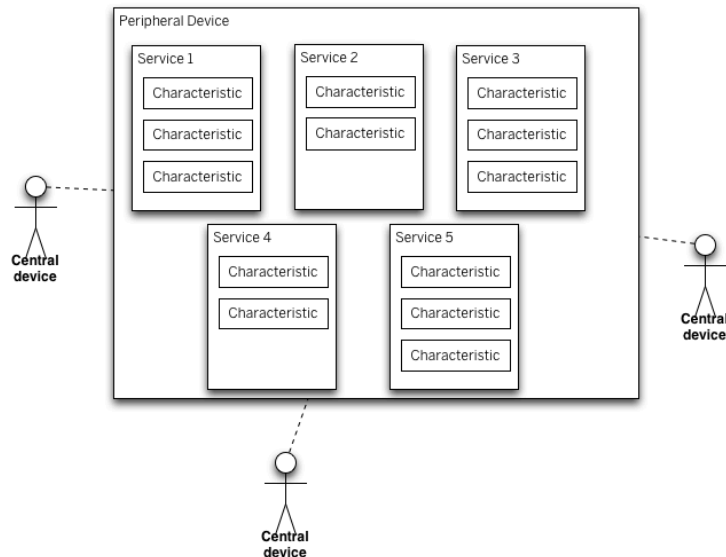
```
#include <ArduinoBLE.h>
```

A quick introduction to BLE

Bluetooth® 4.0 includes both traditional Bluetooth®, now labeled "Bluetooth® Classic", and the Bluetooth® Low Energy. Bluetooth® Low Energy is optimized for low power use at low data rates, and was designed to operate from simple lithium coin cell batteries.

Unlike standard Bluetooth® communication basically based on an asynchronous serial connection (UART) a Bluetooth® LE radio acts like a community bulletin board. The computers that connect to it are like community members that read the bulletin board. Each radio acts as either the bulletin board or the reader. If your radio is a bulletin board (called a peripheral device in Bluetooth® LE parlance) it posts data for all radios in the community to read. If your radio is a reader (called a central device in Bluetooth LE terms) it reads from any of the bulletin boards (peripheral devices) that have information about which it cares. You can also think of peripheral devices as the servers in a client-server transaction, because they contain the information that reader radios ask for. Similarly, central devices are the clients of the Bluetooth® LE world because they read information available from the peripherals.

[Help](#)



Think of a Bluetooth® LE peripheral device as a bulletin board and central devices as viewers of the board. Central devices view the services, get the data, then move on. Each transaction is quick (a few milliseconds), so multiple central devices can get data from one peripheral.

The information presented by a peripheral is structured as **services**, each of which is subdivided into **characteristics**. You can think of services as the notices on a bulletin board, and characteristics as the individual paragraphs of those notices. If you're a peripheral device, you just update each service characteristic when it needs updating and don't worry about whether the central devices read them or not. If you're a central device, you connect to the peripheral then read the boxes you want. If a given characteristic is readable and writable, then the peripheral and central can both change it.

Notify

The Bluetooth® LE specification includes a mechanism known as **notify** that lets you know when data's changed. When notify on a characteristic is enabled and the sender writes to it, the new value is automatically sent to the receiver, without the receiver explicitly issuing a read command. This is commonly used for streaming data such as accelerometer or other sensor readings. There's a variation on this specification called **indicate** which works similarly, but in the indicate specification, the reader sends an acknowledgment of the pushed data.

The client-server structure of Bluetooth® LE, combined with the notify characteristic, is generally called a **publish-and-subscribe model**.

Update a characteristic

Your peripheral should update characteristics when there's a significant change to them. For example, when a switch changes from off to on, update its characteristic. When an analog sensor changes by a significant amount, update its characteristic.

Just as with writing to a characteristic, you could update your characteristics on a regular interval. but this wastes processing power and

Central and Peripheral Devices

Central devices are **clients**. They read and write data from peripheral devices. **Peripheral** devices are **servers**. They provide data from sensors as readable characteristics, and provide read/writable characteristics to control actuators like motors, lights, and so forth.

Services, characteristics, and UUIDs

A Bluetooth® Low Energy peripheral will provide **services**, which in turn provide **characteristics**. You can define your own services, or use standard services (see section 3.4 in the [Assigned Numbers document](#)).

Services are identified by unique numbers known as UUIDs. You know about UUIDs from other contexts. Standard services have a 16-bit UUID and custom services have a 128-bit UUID. The ability to define services and characteristics depends on the radio you're using and its firmware.

Service design patterns

A characteristic value can be up to 512 bytes long. This is a key constraint in designing services. Given this limit, you should consider how best to store data about your sensors and actuators most effectively for your application. The simplest design pattern is to store one sensor or actuator value per characteristic, in ASCII encoded values.

Characteristic	Value
Accelerometer X	200
Accelerometer Y	134
Accelerometer Z	150

This is also the most expensive in memory terms, and would take the longest to read. But it's the simplest for development and debugging.

You could also combine readings into a single characteristic, when a given sensor or actuator has multiple values associated with it.

Characteristic	Value
Motor Speed, Direction	150,1
Accelerometer X, Y, Z	200,133,150

This is more efficient, but you need to be careful not to exceed the 512-byte limit. The accelerometer characteristic above, for example, takes 11 bytes as an ASCII-encoded string.

Read/write/notify/indicate

There are 4 things a central device can do with a characteristic:

Read: ask the peripheral to send back the current value of the characteristic. Often used for characteristics that don't change very

Write: modify the value of the characteristic. Often used for things that are like commands, for example telling the peripheral to turn a motor on or off.

Indicate and **Notify:** ask the peripheral to continuously send updated values of the characteristic, without the central having to constantly ask for it.

Advertising and GAP

BLE devices let other devices know that they exist by advertising using the **General Advertising Profile (GAP)**. Advertising packets can contain a device name, some other information, and also a list of the services it provides.

Advertising packets have a limited size. You will only be able to fit a single 128-bit service UUID in the packet. Make sure the device name is not too long, or you won't even be able to fit that.

You can provide additional services that are not advertised. Central devices will learn about these through the connection/bonding process. Non-advertised services cannot be used to discover devices, though. Sometimes this is not an issue. For example, you may have a custom peripheral device with a custom service, but in your central device app you may know that it also provides the Battery Service and other services.

GATT

The Bluetooth LE protocol operates on multiple layers. **General Attribute Profile (GATT)** is the layer that defines services and characteristics and enables read/write/notify/indicate operations on them. When reading more about GATT, you may encounter GATT concepts of a "server" and "client". These don't always correspond to central and peripherals. In most cases, though, the peripheral is the GATT server (since it provides the services and characteristics), while the central is the GATT client.

Library structure

As the library enables multiple types of functionality, there are a number of different classes.

`BLE` used to enable the Bluetooth® Low Energy module.

`BLEDevice` used to get information about the devices connected or discovered while scanning.

`BLEService` used to enable the services board provides or interact with services a remote board provides.

`BLECharacteristic` used to enable the characteristics board offers in a service or interact with characteristics a remote board provides.

`BLEDescriptor` used to describe a characteristic the board offers.

Used to describe a characteristic the board offers

▼ BLEDescriptor()

Create a new Bluetooth® Low Energy descriptor.

Syntax

```
1 BLEDescriptor(uuid, value, valueSize)
2 BLEDescriptor(uuid, stringValue)
```

Parameters

uuid: 16-bit or 128-bit UUID in string format

value: byte array value

valueSize: size of byte array value

stringValue: value as a string

Returns

New **BLEDescriptor** with the specified **UUID** and value

Example

```
1 BLEDescriptor millisLabelDescriptor("2901", "milli:");
```

▼ bleDescriptor.uuid()

Query the UUID of the specified BLEDescriptor.

Syntax

```
1 bleDescriptor.uuid()
```

Parameters

None

Returns

UUID of the Bluetooth® Low Energy descriptor (as a String).

Example

```
1 BLEDescriptor millisLabelDescriptor("2901", "milli");
2
3
4 Serial.print("millis label descriptor UUID = ");
5 Serial.println(millisLabelDescriptor.uuid());
```

▼ bleDescriptor.valueSize()

Query the value size of the specified BLEDescriptor.

Syntax

```
1 bleDescriptor.valueSize()
```

Parameters

None

Returns

Value size (in bytes) of the Bluetooth® Low Energy descriptor.

Example

```
1 BLEDescriptor millisLabelDescriptor("2901", "milli-  
2  
3  
4 Serial.print("millis label descriptor value size =  
5 Serial.println(millisLabelDescriptor.valueSize());
```

▼ bleDescriptor.valueLength()

Query the length, in bytes, of the descriptor current value.

Syntax

```
1 bleDescriptor.valueLength()
```

Parameters

None

Returns

Length of descriptor value in bytes.

Example

```
1 // read the descriptor value  
2 descriptor.read();  
3  
4 // print out the value of the descriptor  
5 Serial.print(", value 0x");  
6 printData(descriptor.value(), descriptor.valueLength());  
7  
8 // ...  
9  
10 void printData(const unsigned char data[], int length) {  
11   for (int i = 0; i < length; i++) {  
12     unsigned char b = data[i];  
13  
14     if (b < 16) {  
15       Serial.print("0");  
16     }  
17  
18     Serial.print(b, HEX);  
19   }  
20 }
```

▼ bleDescriptor.value()

Syntax

```
1 bleDescriptor.value()
```

Parameters

None

Returns

Value byte array of the **BLE descriptor**.

Example

```
1 BLEDescriptor millisLabelDescriptor("2901", "milli")
2
3
4
5 int descriptorValueSize = millisLabelDescriptor.valueSize();
6 byte descriptorValue[descriptorValueSize];
7
8 for (int i = 0; i < descriptorValueSize; i++) {
9     descriptorValue[i] = millisLabelDescriptor.value[i];
10 }
```

▼ bleDescriptor.readValue()

Read the current value of the descriptor. If the descriptor is on a remote device, a read request will be sent.

Syntax

```
1 bleDescriptor.readValue(buffer, length)
2 bleDescriptor.readValue(value)
```

Parameters

buffer: byte array to read value into

length: size of buffer argument in bytes

value: variable to read value into (by reference)

Returns

Example

```
1 byte value = 0;
2
3 // get the value, descriptor is 1 byte so use by
4 descriptor.readValue(value);
```

▼ bleDescriptor.read()

Perform a read request for the descriptor.

Syntax

```
1 bleDescriptor.read()
```

Parameters

None

Returns

true, if successful,

false on failure

Example

```
1 if (descriptor.read()) {
2     Serial.println("descriptor value read");
3
4     // ...
5 } else {
6     Serial.println("error reading descriptor value");
7 }
```

Was this article helpful?



Connect and Contribute

